CS151 Intro to Data Structures

Heaps II

Announcements

HW5 and Lab7 due Sunday

Lab today: BinaryTree implementation

- autograded
- submitted as part of HW5

Outline

- Heap Review
- Selection sort and heap sort
- Array based binary trees

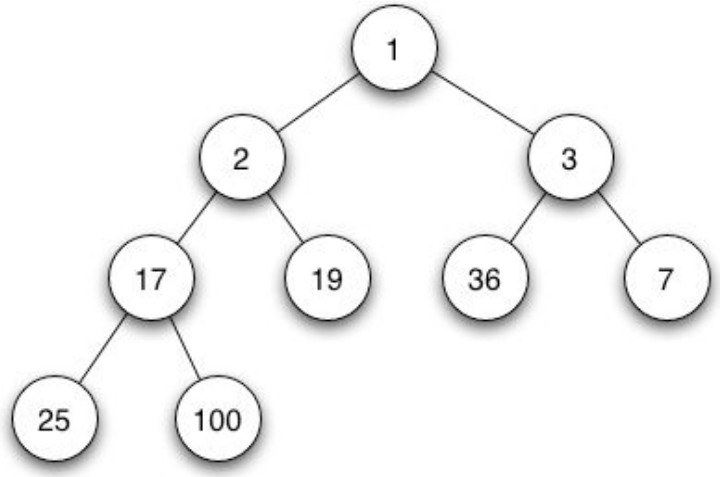
Heap Review

Binary Heap Properties

1. Each node has **at most 2 children**

1. For every node n (except for the root): **n.key >= parent(n).key**

1. **Complete:** all levels of the tree, except possibly the last one are fully filled, and, if the last level of the tree is not complete, the nodes of that level are filled from left to right



Heap Operations:

1. Insert
   a. Upheap
2. Poll
   a. Downheap
3. Search
4. Remove

Warmup

Insert the following keys one at a time into an initially empty min-heap:

15 3 8 1 12 6 9 4

run the following operations:

- poll()
- insert(2)
- poll()

Selection Sort

In place sorting algorithm

1. Separate the array into “sorted” and “unsorted”
   a. sorted starts empty
2. Find the min element in the unsorted array
3. Swap min with the first element in unsorted
4. repeat

Selection Sort

code

Runtime complexity?

O(n^2)

Space complexity?

O(1)

Heap Sort

Heap Sort

- “selection sort with the correct data structure”

- divides input into a sorted and an unsorted region
- iteratively shrinks the unsorted region by extracting the min element from it and inserting it into the sorted region

Heap Sort

- “selection sort with the correct data structure”

Expensive portion of selection sort?

Heap sort: instead of looping over the unsorted portion of the array, store the unsorted data in a heap!

Now the min is always at the top.

Runtime complexity of poll? O(logn)

Heap Sort

1. Heap construction
   a. rearrange the array into a heap
2. Heap extraction
   a. iteratively **poll** and insert into the sorted portion

Heap Sort - Example

[64, 25, 12, 22, 11]

Heap Sort

1. Heap Construction Phase:
   a. runtime complexity?

2. Heap Extraction Phase:
   b. runtime complexity?

Overall Runtime Complexity?

Space complexity?

Heap Sort

O(nlogn) sort algorithm

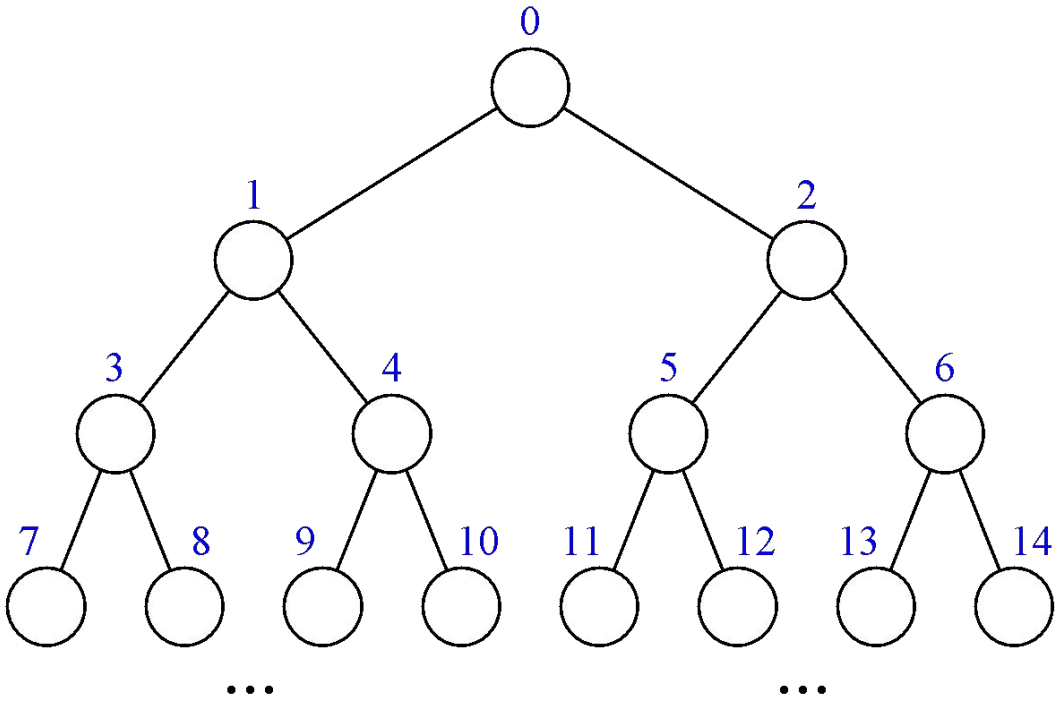
Selection sort with the right data structure

Array-based Implementation

- BinaryTrees can be implemented in different ways
  - Linked nodes - what you'll do in your homework
  - Array

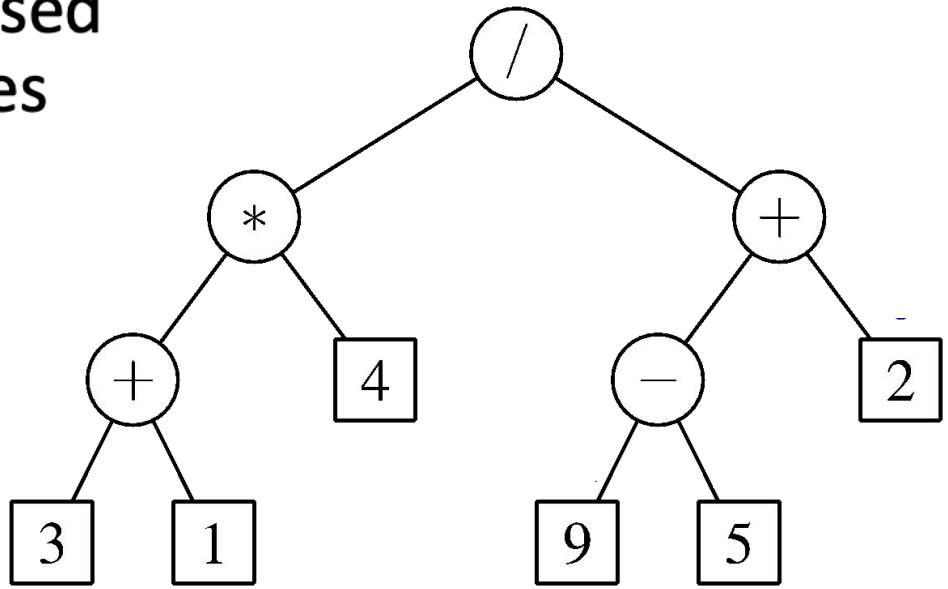
Array-based Implementation

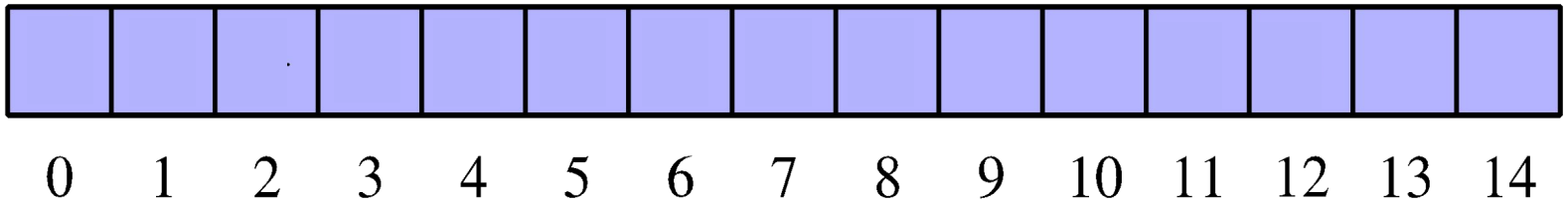
- Number nodes level-by-level, left-to-right
- $f(root) = 0$
- $f(l) = 2f(p) + 1$
- $f(r) = 2f(p) + 2$
- Numbering is based on all positions, not just occupied positions



Array-based Binary Tree

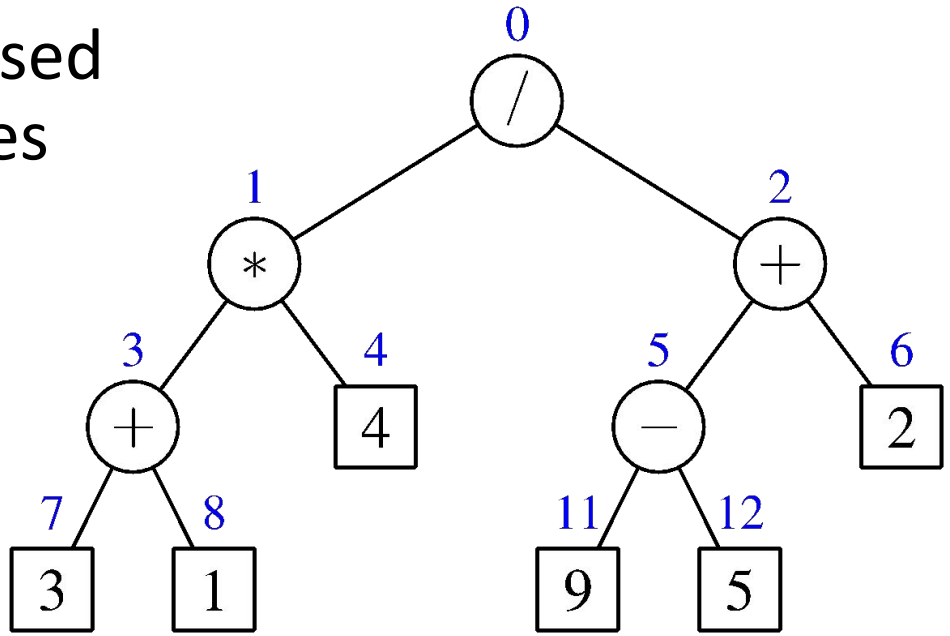
- The numbering can then be used as indices for storing the nodes directly in an array
- $f(root) = 0$
- $f(l) = 2f(p) + 1$
- $f(r) = 2f(p) + 2$





Array-based Binary Tree

- The numbering can then be used as indices for storing the nodes directly in an array

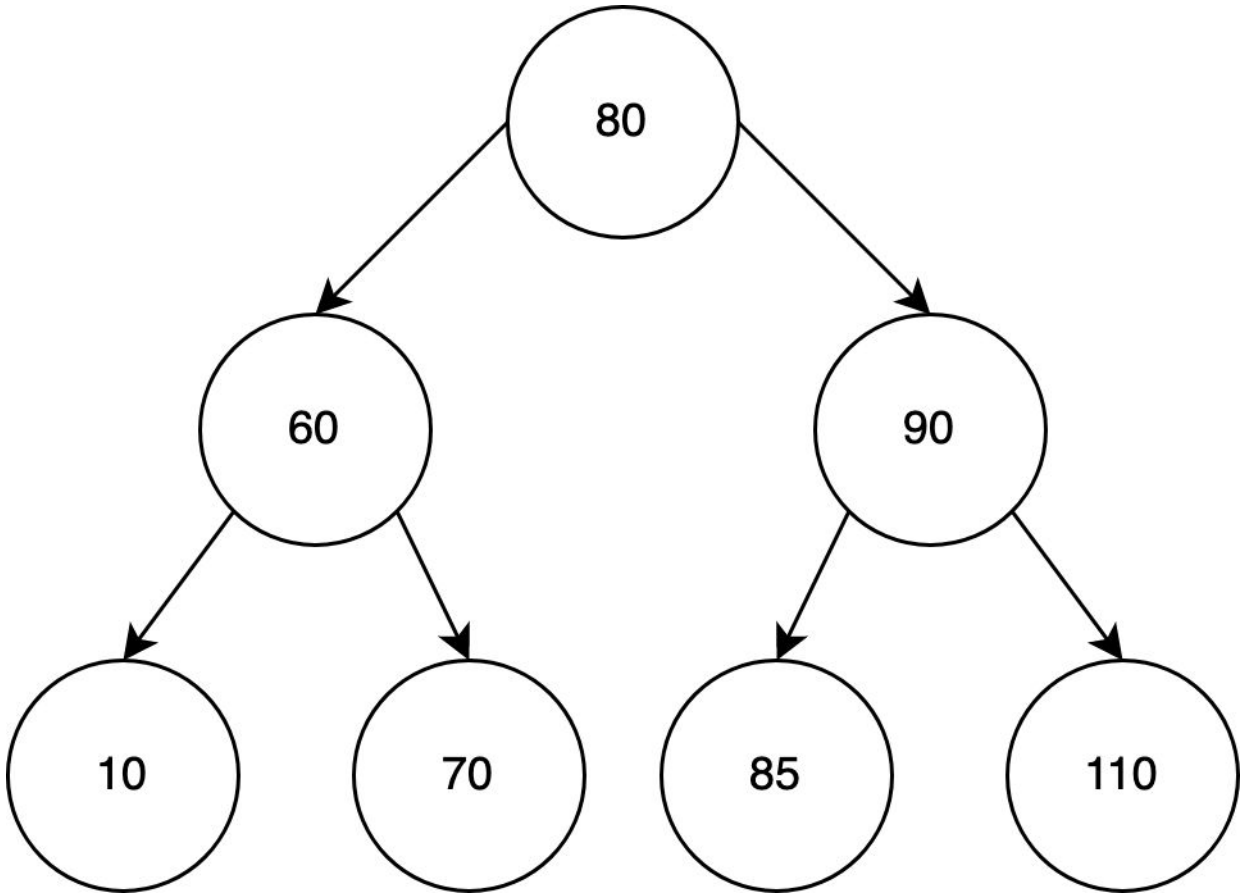


/	*	+	+	4	−	2	3	1			9	5		
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Array-based Binary Tree

What would the underlying array look like for this tree?

```
f(root)  = 0
f(l)     = 2*f(p) + 1
f(r)     = 2*f(p) + 2
```

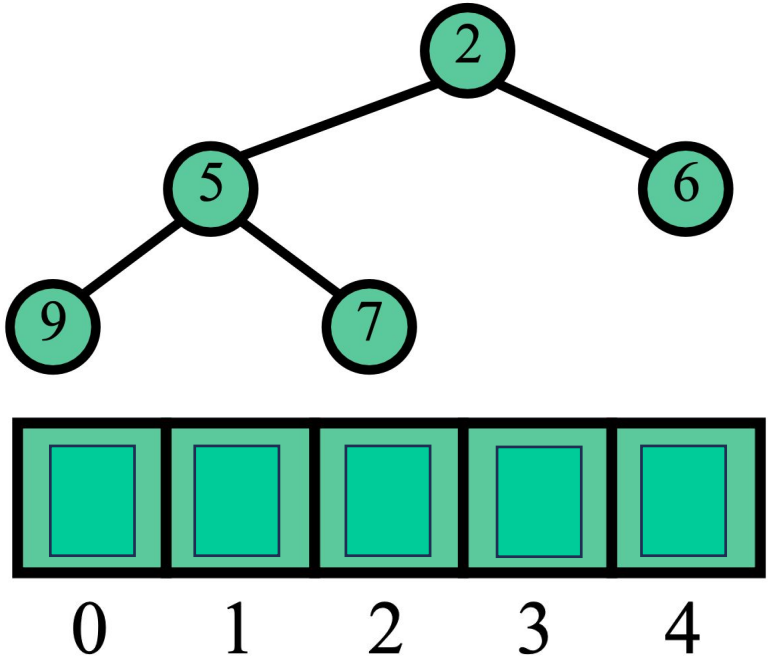


Now, let's insert 75. Where should it go?

Array based heap

Node at index i

- Left child index:
  - 2i + 1
- Right child index:
  - 2i + 2

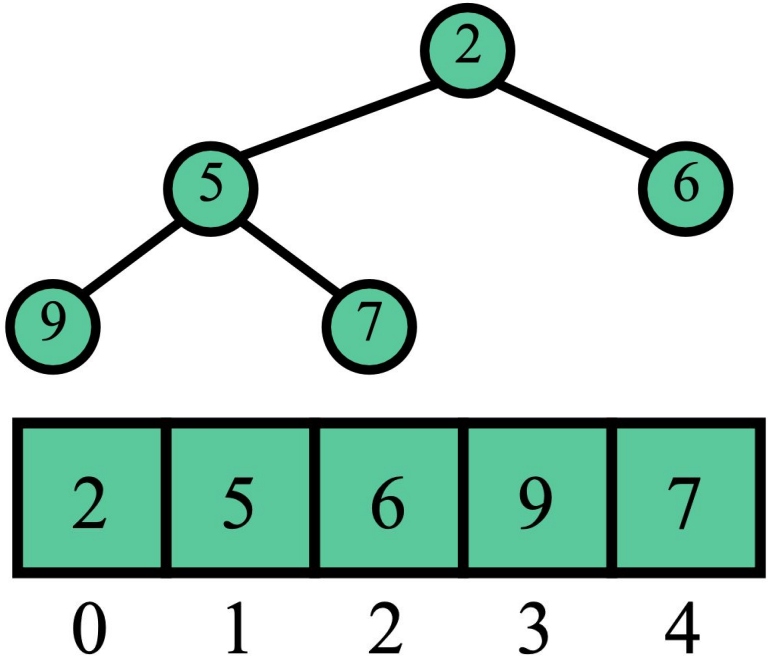


Array based heap

Node at index i

- Left child index:
  - 2i + 1
- Right child index:
  - 2i + 2

- Peek:
  - Get element at index 0
- Poll:
  - Remove element at index 0
- No need to store references/links



Repeat the warmup with an array heap

Insert the following keys one at a time into an initially empty min-heap:

15 3 8 1 12 6 9 4

run the following operations:

- poll()
- insert(2)
- poll()

Summary

1. Heaps
   a. DS for efficient removal of the min element
2. Selection Sort
   a. O(n^2) sort algorithm
3. Heap Sort
   a. O(nlogn) sort algorithm